



**UNIVERSIDAD UNIVERSIDAD TÉCNICA ESTATAL DE
QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN**

SISTEMAS OPERATIVOS

MÉTODOS HASH

INTEGRANTES:

DÍAZ PONTÓN STEVEN

GAMARRA ZÁRATE JAMILETH

HERRERA RAMOS THAIS

TIGASI SAMPEDRO PAÚL

TRUJILLO VEGA MAYUMMY

CARRERA/CURSO:

SOFTWARE 4TO "A"

DOCENTE:

ING. PORTILLA OLVERA GILBERTO ELIAS

1. Explique el concepto de resistencia a preimagen (o a primera preimagen) y su importancia en la seguridad de los datos.

Definición

La resistencia a primera preimagen es la propiedad de una función hash H que garantiza que, conociendo únicamente un valor resumen (hash) h , es computacionalmente inviable encontrar cualquier entrada m tal que $H(m) = h$. En otras palabras, el proceso de cálculo del hash debe ser de una sola dirección: fácil de aplicar hacia adelante, pero prácticamente imposible de invertir [1].

Ejemplo práctico

Un caso real es el de los repositorios de descarga de software libre. Cuando una distribución Linux publica su imagen ISO, junto a ella se publica un archivo con el hash SHA-256 de esa imagen. Si un atacante intercepta ese archivo de hashes en tránsito, no puede reconstruir la ISO completa (varios gigabytes de datos) a partir del valor resumen de 256 bits; el hash no contiene información suficiente para reconstruir la entrada [1], [2].

Imagen ISO (2 GB) ----- $H(x) = \text{SHA-256}$ ----- 9f86d081884c7d65... (32 bytes)
--

Con el hash publicado NO es posible reconstruir los 2 GB originales.
--

Importancia en seguridad

Esta propiedad es la base de cualquier esquema que use hashes para proteger datos sensibles (contraseñas, claves privadas, tokens). Si un algoritmo careciera de resistencia a primera preimagen, cualquier valor hash filtrado o interceptado equivaldría a exponer directamente el dato original, eliminando por completo la protección que se buscaba ofrecer [1].

2. ¿En qué consiste el atributo "resistencia a colisiones" de un algoritmo hash?

Definición del atributo

Consiste en la dificultad computacional de encontrar dos entradas distintas, m_1 diferente de m_2 , que produzcan exactamente el mismo valor hash: $H(m_1) = H(m_2)$. A esa coincidencia se le llama colisión, y un buen algoritmo hash debe hacerla extremadamente improbable de generar de forma deliberada [3], [4].

Ejemplo de colisión

Un caso documentado ocurrió en 2012 con el malware Flame, que logró falsificar un certificado digital de Microsoft explotando una colisión en el algoritmo MD5. Los atacantes generaron un certificado fraudulento cuyo hash MD5 coincidía con el de un certificado legítimo firmado por Microsoft, lo que permitió que el malware se hiciera pasar por software confiable y se distribuyera como si fuera una actualización oficial [5].

Certificado legítimo ----- MD5 ----- 7a4e9c...
--

Certificado falsificado ----- MD5 ----- 7a4e9c... (misma clave)

El sistema valida el hash, no el contenido, ambos certificados 'pasan' la verificación, aunque uno sea fraudulento.

Análisis de impacto en seguridad

Cuando un algoritmo hash pierde resistencia a colisiones, un atacante puede sustituir un archivo, contrato o certificado legítimo por uno malicioso sin que los mecanismos de verificación detecten el cambio, ya que estos solo comparan el valor resumen. Por esta razón MD5 y SHA-1 fueron retirados progresivamente de los estándares de firma digital y certificados TLS [1], [6], [7].

3. Explique el concepto de resistencia a segunda preimagen y su importancia en la seguridad de los datos.

Definición precisa

Dado un mensaje específico m_1 y su hash $H(m_1)$, la resistencia a segunda preimagen garantiza que es computacionalmente inviable encontrar un mensaje diferente m_2 (m_2 distinto de m_1) que produzca el mismo resultado: $H(m_2) = H(m_1)$ [1].

Diferencia con primera preimagen

La diferencia está en el punto de partida del ataque. En primera preimagen, el atacante solo dispone del hash y busca cualquier entrada que lo genere, sin conocer ningún mensaje original. En segunda preimagen, el atacante ya posee un mensaje real y legítimo, y su objetivo es fabricar una alternativa distinta que produzca ese mismo hash para sustituirla sin levantar sospechas; es un ataque más dirigido y con un objetivo concreto [3]. Un buen algoritmo hash impide esto reforzando el llamado efecto avalancha: incluso un cambio mínimo en la entrada debe producir un hash de salida que parezca completamente distinto y no correlacionado con el original [8].

Implicaciones prácticas

Un ejemplo aplicado es el de las facturas electrónicas firmadas digitalmente: si un contador envía una factura por 500 dólares y esta se firma calculando su hash, la resistencia a segunda preimagen impide que alguien intercepte esa factura y la reemplace por otra con un monto de 5000 dólares que, por coincidencia del algoritmo, produzca el mismo hash y por tanto pase la validación de la firma como si fuera el documento original [9].

Factura original: 'Monto: \$500' ---- $H(x)$ ---- c1f2a9...
Factura falsa: 'Monto: \$5000' ---- $H(x)$ ---- debe dar un valor TOTALMENTE distinto (efecto avalancha)

4. Analice cómo la longitud de salida fija de un hash influye en la probabilidad de colisiones.

Explicación de la longitud de salida

Toda función hash produce una salida de longitud fija, sin importar el tamaño de la entrada: un mensaje de una palabra y un archivo de varios gigabytes generan resúmenes del mismo tamaño en bits. Esa longitud fija define el número total de valores hash distintos que el algoritmo puede producir [1].

Relación con la probabilidad de colisiones

Como las entradas posibles son infinitas pero las salidas posibles son un número finito (2 elevado a n , siendo n la cantidad de bits de salida), las colisiones son matemáticamente inevitables tarde o temprano (principio del palomar). Sin embargo, mientras mayor sea n , mayor es el espacio de valores disponibles y más costoso resulta para un atacante encontrar una colisión por fuerza bruta [10].

Ejemplo numérico comparativo

MD5 -> 128 bits -> $2^{128} \approx 3.4 \times 10^{38}$ valores posibles
SHA-256 -> 256 bits -> $2^{256} \approx 1.1 \times 10^{77}$ valores posibles
SHA-512 -> 512 bits -> $2^{512} \approx 1.3 \times 10^{154}$ valores posibles
Duplicar la longitud de salida no duplica la seguridad: la multiplica exponencialmente.

Por esta razón MD5 (128 bits) ya es inviable para uso criptográfico, mientras que SHA-256 y SHA-512 se mantienen como estándares seguros en aplicaciones como blockchain y protocolos TLS actuales [1], [2].

5. Describa el papel de las funciones hash en la integridad de mensajes transmitidos por redes inseguras.

Definición de integridad de mensajes

La integridad de un mensaje es la garantía de que su contenido no ha sido alterado, ya sea de forma accidental (errores de transmisión) o deliberada (manipulación de un atacante), entre el momento en que fue enviado y el momento en que es recibido [4], [11].

Rol de las funciones hash en garantizarla

El emisor calcula el hash del mensaje original antes de enviarlo. Al llegar al destino, el receptor vuelve a calcular el hash del mensaje recibido y lo compara con el valor original. Si ambos coinciden, se confirma que el contenido no cambió durante el trayecto por la red, incluso si esa red (una conexión wifi-pública, por ejemplo) no es confiable [4].

Ejemplo práctico

En las transferencias bancarias por internet, cada operación viaja junto con un código de verificación derivado de un hash del mensaje (monto, cuenta origen, cuenta destino). Si un atacante en la misma red wifi intercepta y altera el monto de la transacción, el hash recalculado en el servidor del banco no coincidirá con el hash recibido, y la operación será rechazada automáticamente antes de procesarse [11].

6. Explique por qué las funciones hash se consideran unidireccionales y cómo esto protege la información original.

Explicación del concepto

La unidireccionalidad no es una limitación de diseño, sino una consecuencia directa de cómo operan internamente los algoritmos hash. Un dato de entrada, sin importar su tamaño, se procesa mediante múltiples rondas de operaciones matemáticas desplazamientos de bits, sumas modulares y funciones lógicas como AND, OR y XOR combinadas de forma no lineal que comprimen esa entrada hasta un resultado de longitud fija.

El punto clave es que varias de esas operaciones internas descartan información en cada ronda: por ejemplo, un desplazamiento de bits hacia la derecha (shift right) elimina permanentemente los bits que salen por el extremo, y una operación XOR entre dos rotaciones puede producir el mismo resultado a partir de combinaciones de entrada distintas, por lo que ya no es posible determinar con certeza cuál fue la combinación original.

Al repetirse esto en decenas de rondas 64 en el caso de SHA-256, el resultado final ya no conserva rastros suficientes del dato original como para reconstruirlo mediante ningún procedimiento matemático directo; la única alternativa que le queda a un atacante es probar entradas por fuerza bruta hasta encontrar, por coincidencia, una que genere el mismo resultado [16].

Relación con la protección de datos

Un hash puede tratarse como sustituto seguro del dato real: no es que esté "escondido" o cifrado con una clave que alguien pueda usar para revertirlo, es que el proceso que lo generó pierde información de forma irreversible en cada paso. Esto lo distingue de la encriptación convencional, donde sí existe una clave capaz de deshacer la transformación; en el hash no existe absolutamente ninguna clave de reversión, porque el algoritmo nunca fue construido para ser invertido, solo para ser comparado.

Ejemplo de aplicación

Un caso llamativo es la generación de direcciones de billeteras de Bitcoin. Cada dirección pública que se comparte para recibir fondos no es la clave pública del usuario, sino el resultado de aplicarle dos funciones hash consecutivas (SHA-256 y RIPEMD-160). Cualquier persona puede consultar esa dirección en la blockchain, que es completamente pública, pero nadie puede revertir el proceso para obtener la clave pública original y mucho menos la clave privada, que es la que realmente controla los fondos. Aquí la unidireccionalidad es la que separa lo que puede exponerse abiertamente de lo que debe permanecer protegido, incluso dentro de un sistema diseñado para ser transparente y descentralizado.

7. Identifique un ejemplo práctico de uso de funciones hash en el almacenamiento de contraseñas y discuta sus ventajas.

Ejemplo de uso en contraseñas

Cuando un usuario crea una cuenta, el sistema no guarda su contraseña en texto plano: calcula su hash (idealmente con un algoritmo pensado para contraseñas, como bcrypt o Argon2) y almacena únicamente ese resumen. En cada inicio de sesión, se hashea la contraseña ingresada y se compara con el valor guardado; si coinciden, se concede el acceso [12].

Ventajas frente al almacenamiento en texto plano

Si la base de datos es robada, el atacante obtiene solo los hashes y no las contraseñas reales, por lo que debe intentar revertirlos mediante fuerza bruta o diccionarios, un proceso costoso en tiempo y recursos. Esto protege tanto a la empresa como a los usuarios, especialmente porque muchas personas reutilizan la misma contraseña en distintos servicios [13].

Riesgos y medidas adicionales

Un caso ilustrativo fue la filtración de LinkedIn en 2012, donde se descubrió que la plataforma almacenaba contraseñas con SHA-1 sin agregar una sal (salt) aleatoria, lo que permitió a los atacantes usar tablas precalculadas (rainbow tables) para revertir millones de hashes en poco tiempo [14]. Por eso hoy se recomienda combinar cada contraseña con una sal única por usuario y utilizar algoritmos intencionalmente lentos (bcrypt, scrypt, Argon2), que encarecen los ataques de fuerza bruta incluso si la base de datos es comprometida [13], [15].

8. Compare la diferencia entre un hash criptográfico seguro y un hash no diseñado para seguridad (por ejemplo, CRC).

Diferencia conceptual

Un hash criptográfico seguro, como SHA-256, está diseñado explícitamente para resistir a un adversario que intenta manipular deliberadamente el sistema, cumpliendo las propiedades de resistencia a preimagen, segunda preimagen y colisiones. Un hash no criptográfico, en cambio, se diseñó únicamente para detectar errores accidentales de transmisión o almacenamiento (ruido, corrupción de bits), no para resistir ataques intencionados [4].

La siguiente tabla resume las diferencias principales entre ambos tipos de algoritmo:

Característica	CRC-32 (hash no criptográfico)	SHA-256 (hash criptográfico seguro)
Propósito de diseño	Detectar errores accidentales de transmisión o almacenamiento.	Resistir la manipulación intencional por parte de un atacante.
Tamaño de salida	32 bits.	256 bits.
Naturaleza matemática	Función lineal basada en división polinomial.	Función no lineal con efecto avalancha.
Reversibilidad / manipulación	Un atacante puede recalcular fácilmente un valor que produzca el mismo CRC tras modificar los datos.	Es computacionalmente inviable manipularlo o revertirlo para obtener el mismo hash.
Resistencia a colisiones	Baja; las colisiones se generan con facilidad debido al espacio reducido de 2^{32} valores.	Alta; encontrar una colisión requiere un esfuerzo computacional impracticable.
Uso típico	Archivos ZIP, imágenes PNG y verificación de paquetes en protocolos de red.	Almacenamiento de contraseñas, firmas digitales, certificados y verificación de descargas de software.

Ejemplo de hash no seguro

El CRC-32, usado comúnmente en archivos ZIP y en protocolos de red para verificar que un paquete no se corrompió en el trayecto, es un ejemplo típico de hash no criptográfico. Al ser una operación matemáticamente lineal, un atacante puede calcular fácilmente un ajuste al archivo modificado que produzca el mismo valor de CRC que el original, algo impensable con SHA-256 [4]. Este tipo de algoritmos fueron de los primeros en descartarse para fines de seguridad precisamente porque nunca fueron concebidos para resistir un ataque deliberado, sino solo para detectar corrupción accidental.

Análisis de riesgos

Usar CRC-32 (o algoritmos similares) para verificar contraseñas, firmas digitales o la integridad de una actualización de software sería gravemente inseguro: protegería contra errores aleatorios de transmisión,

pero no contra un atacante que modifique el contenido a propósito, ya que puede recalcular fácilmente un CRC coincidente sin necesidad de romper ninguna propiedad criptográfica real [4]. Por esta razón, cualquier aplicación que dependa de la confianza frente a un adversario activo debe emplear exclusivamente algoritmos hash criptográficos reconocidos.

9. Analice el impacto de los ataques de cumpleaños en la resistencia a colisiones de un algoritmo hash.

Definición clara del ataque

El ataque de cumpleaños es una técnica que permite encontrar dos entradas distintas que generen el mismo hash (una colisión) con muchos menos intentos de los que se pensaría necesarios. En lugar de intentar adivinar qué entrada produce un hash específico ya conocido, el atacante genera muchas entradas al azar y las va comparando entre sí, buscando que dos cualquiera coincida en su resultado; esto reduce drásticamente el número de pruebas necesarias frente a un ataque de fuerza bruta tradicional [11].

Relación con la paradoja del cumpleaños

Su nombre proviene de la paradoja del cumpleaños: en un grupo de apenas 23 personas, la probabilidad de que dos compartan la misma fecha de nacimiento supera el 50%, aunque existen 365 días posibles. Esto ocurre porque lo que se compara no son pares aislados, sino todas las combinaciones posibles entre los miembros del grupo, que crecen mucho más rápido que el grupo mismo [3].

Reducción del número de intentos

Hash de n bits:

Buscar UNA colisión con una entrada específica --- $\approx 2^n$ intentos

Buscar CUALQUIER par que coincida (cumpleaños) --- $\approx 2^{(n/2)}$ intentos

Implicaciones prácticas

Esto significa que la seguridad real de un hash frente a colisiones es la mitad de su longitud nominal en bits. Un algoritmo de 128 bits, que en teoría ofrece 2^{128} de seguridad, en la práctica solo brinda unos 2^{64} de resistencia efectiva ante colisiones, una cifra que con suficiente poder de cómputo distribuido puede volverse alcanzable, como ocurrió con ataques prácticos contra MD5 y, más tarde, contra SHA-1 [6], [7].

Medida de mitigación

La mitigación principal es utilizar algoritmos con una longitud de salida suficientemente grande. Con SHA-256, el ataque de cumpleaños reduce la seguridad a 2^{128} intentos, una cifra que sigue siendo computacionalmente inviable con la tecnología actual, razón por la cual se prefieren longitudes de 256 bits o más en aplicaciones críticas [10].

Referencias

- [1] National Institute of Standards and Technology, "Secure hash standard (SHS)," Federal Inf. Process. Standards Publ. FIPS PUB 180-4, Gaithersburg, MD, USA, Aug. 2015.
- [2] R. Rivest, "The MD5 message-digest algorithm," IETF, RFC 1321, Apr. 1992.
- [3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, Handbook of Applied Cryptography. Boca Raton, FL, USA: CRC Press, 1996.
- [4] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Boston, MA, USA: Pearson, 2017.
- [5] Microsoft Security Response Center, "Flame malware collision attack explained," Microsoft MSRC Blog, Jun. 6, 2012.
- [6] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, "Announcing the first SHA1 collision," Google Security Blog, Feb. 23, 2017.
- [7] National Institute of Standards and Technology, "Research results on SHA-1 collisions," NIST Comput. Security Resource Center, Feb. 2017.

- [8] National Institute of Standards and Technology, "Public comments on FIPS 180-4, Secure Hash Standard (SHS)," NIST Comput. Security Resource Center, Gaithersburg, MD, USA, 2022.
- [9] National Institute of Standards and Technology, "Digital signature standard (DSS)," Federal Inf. Process. Standards Publ. FIPS PUB 186-5, Gaithersburg, MD, USA, Feb. 2023.
- [10] National Institute of Standards and Technology, "Recommendation for applications using approved hash algorithms," NIST Special Publ. 800-107, Gaithersburg, MD, USA, 2009.
- [11] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, RFC 2104, Feb. 1997.
- [12] Open Worldwide Application Security Project, "Password storage cheat sheet," OWASP Cheat Sheet Series, 2024.
- [13] N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. USENIX Annu. Tech. Conf. (ATEC '99), Monterey, CA, USA, Jun. 1999, pp. 81–91.
- [14] B. Krebs, "As scope of 2012 breach expands, LinkedIn to again reset passwords for some users," Krebs on Security, May 18, 2016.
- [15] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," IETF, RFC 9106, Sep. 2021.
- [16] A. (akira_19), "Why is a Hash Function Irreversible?," DEV Community, Nov. 16, 2024.